Security Implications of LLMs in Web Application Development and Deployment

Author: Charlie Luca, Zaynab Sai'd

Date: February 18, 2024.

ABSTRACT

The integration of Large Language Models (LLMs) into web application development and deployment pipelines is reshaping the software engineering landscape by accelerating code generation, automating documentation, and supporting intelligent decision-making. However, this paradigm shift introduces novel security implications that are not yet fully understood or addressed. This research investigates the multifaceted security risks posed by LLMs in web development environments, including prompt injection attacks, model hallucinations, inadvertent code vulnerabilities, and the leakage of sensitive information. Through a comprehensive threat modeling approach, the study evaluates the attack surfaces introduced by LLM-powered coding assistants, DevOps tools, and API integrations. Furthermore, the paper assesses the adequacy of current guardrail mechanisms, such as input validation, sandboxing, and adversarial testing, in mitigating these threats. A hybrid methodology combining empirical case studies, vulnerability assessments, and stakeholder interviews is employed to derive actionable insights. The findings underscore the urgent need for LLM-aware secure software development practices and propose a risk management framework tailored to AI-augmented web engineering workflows. This work contributes to bridging the gap between AI innovation and cybersecurity resilience in the evolving digital ecosystem.

Keywords

Large Language Models (LLMs); Web Application Security; Prompt Injection; Secure Software Development Lifecycle (SSDLC); Code Vulnerabilities; AI-Augmented Development; Model Hallucination; Threat Modeling; Guardrail Mechanisms; DevSecOps.

I. INTRODUCTION

Background

The advent of Large Language Models (LLMs) such as OpenAI's GPT series and Google's Gemini has transformed the landscape of software engineering, particularly in web application development. These models are now widely adopted for automating tasks including code generation, unit testing, API documentation, and even DevOps workflows. Their natural language understanding and generation capabilities enable developers to accelerate the software development lifecycle (SDLC) by integrating intelligent coding assistants into their daily workflows. As LLMs become embedded in integrated development environments (IDEs), continuous integration/continuous deployment (CI/CD) pipelines, and user-facing web functionalities, their role in modern software engineering is becoming both indispensable and pervasive.

Problem Statement

Despite their benefits, the growing dependence on LLMs introduces novel and underexplored security vulnerabilities across the web development and deployment pipeline. LLMs can generate insecure code, leak sensitive data, and become susceptible to adversarial inputs such as prompt injection or model inversion attacks. Furthermore, the non-deterministic behavior and lack of interpretability in LLM-generated outputs pose challenges for validation and secure integration. These emerging threats challenge traditional assumptions in web application security and call for a reevaluation of secure development practices in AI-augmented environments.

Research Objectives

This research aims to:

1. Identify and classify the security implications associated with the use of LLMs in web application development and deployment.
2. Analyze real-world scenarios where LLM integration has led or could lead to exploitable security flaws.
3. Propose and evaluate technical and procedural strategies for mitigating LLM-induced risks within secure development pipelines.

Research Questions

- RQ1: What are the primary security risks introduced by integrating LLMs in the web application development and deployment process?
- RQ2: How do these risks differ from traditional software vulnerabilities?
- RQ3: What frameworks, tools, or best practices can developers and organizations adopt to mitigate these risks effectively?

Scope and Limitations

This study focuses on the use of general-purpose LLMs in web development contexts, particularly in environments where they are applied to tasks such as code suggestion, testing automation, deployment scripting, and documentation generation. The scope includes analysis of risks across local and cloud-based development environments, CI/CD pipelines, and user-facing applications powered by LLMs. Limitations of this research include the fast-evolving nature of LLM technologies, limited access to proprietary model internals, and the complexity of isolating LLM-specific vulnerabilities from broader system-level threats. As such, the findings are contextualized within current-generation models and may require future updates as the technology matures.

II. LITERATURE REVIEW

LLMs in Software Development

Large Language Models (LLMs) have emerged as transformative tools in software development. Platforms such as GitHub Copilot, powered by OpenAI Codex, offer intelligent code suggestions directly within IDEs, while alternatives like Amazon CodeWhisperer and Tabnine provide similar capabilities tailored to various programming environments. These tools assist developers by auto-generating code, recommending bug fixes, and streamlining repetitive tasks. Their integration into CI/CD workflows and version control systems signifies a shift toward AI-augmented development, where human oversight coexists with machine-generated contributions. While these tools have demonstrated efficiency gains, their security implications remain insufficiently explored in the context of complex, production-level web applications.

Web Application Security Basics

Understanding the security implications of LLMs requires a foundational grasp of web application security principles. The OWASP Top 10 remains the most authoritative framework for classifying critical security risks in web applications. Common threats include Injection Attacks (A01), Broken Authentication (A02), Security Misconfiguration (A05), and Insufficient Logging and Monitoring (A09). These vulnerabilities often serve as entry points for attackers and have historically been exploited in high-profile breaches.

In parallel, the Secure Software Development Lifecycle (SSDLC) emphasizes the incorporation of security at every stage of the development process—from requirements analysis and secure design to coding, testing, and deployment. SSDLC frameworks advocate practices such as threat modeling, static and dynamic code analysis, secure code reviews, and security testing. However, the rise of LLMs introduces new automation layers that may bypass or disrupt traditional SSDLC checkpoints if not properly monitored or validated.

Security Challenges of LLMs

The application of LLMs in coding and web development raises several security concerns:

- Code Quality and Hallucinations: LLMs may generate syntactically correct but insecure or semantically flawed code, especially in response to ambiguous or incomplete prompts. Such hallucinations can introduce logic errors, insecure configurations, or reliance on outdated libraries.
- Injection Vulnerabilities: When used to handle user input or generate queries, LLMs may inadvertently create content vulnerable to classic injection attacks (e.g., SQLi, XSS). Worse, LLM-generated code might unknowingly replicate unsafe coding patterns learned from training data.
- Adversarial Prompting: Attackers can exploit LLM behavior through specially crafted prompts that cause the model to deviate from its intended logic, disclose sensitive information, or perform unintended operations.
- Data Poisoning and Model Exploits: LLMs trained or fine-tuned on unvetted code repositories may inherit vulnerabilities or be exposed to poisoned datasets, potentially embedding exploitable flaws into generated outputs.

Existing Research Gaps

While several studies have analyzed the productivity benefits and accuracy of LLMs in software development, relatively few have examined their security impact in real-world deployment scenarios. Most existing literature focuses on synthetic benchmarks, academic prototypes, or isolated code snippets rather than full-stack, end-to-end web applications. There is also a lack of comprehensive frameworks for integrating LLM-aware security testing into the SSDLC. Furthermore, while early-stage efforts exist to develop LLM guardrails and secure generation filters, there is limited empirical evaluation of their effectiveness in live development environments.

III. METHODOLOGY

Research Design

This study adopts a mixed-methods research design combining both qualitative and quantitative approaches to investigate the security implications of Large Language Models (LLMs) in web application development. The qualitative dimension focuses on gathering expert insights and contextual understanding through interviews and thematic analysis, while the quantitative aspect involves empirical security audits, vulnerability scans, and experimental testing of LLM-generated outputs. This dual approach ensures both depth and breadth in identifying and evaluating the associated risks.

Data Collection

To ground the research in real-world practices, data will be collected through multiple sources:

- Case Studies: Detailed examinations of web applications developed or significantly augmented using LLM-based tools such as GitHub Copilot, OpenAI Codex, and CodeWhisperer. These case studies will focus on end-to-end development workflows, including front-end, back-end, and API layers.
- Security Audits: Static and dynamic code analysis will be performed on LLM-assisted development projects. The focus will be on identifying vulnerabilities such as injection flaws, misconfigurations, insecure authentication mechanisms, and use of deprecated libraries.
- Interviews: Semi-structured interviews will be conducted with software developers, DevOps engineers, and security professionals who have experience with LLM-integrated development environments. The aim is to understand common security concerns, mitigation strategies, and perceived gaps in current toolchains.

Risk Analysis Framework

The study will employ established threat modeling techniques to assess LLM-induced risks:

- STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege) will be used to categorize threats introduced during development and deployment phases.
- Alternatively, the DREAD model (Damage potential, Reproducibility, Exploitability, Affected users, and Discoverability) will be applied to quantify and prioritize vulnerabilities.
- Automated Code Scanning Tools: Tools such as SonarQube, Checkmarx, and Static Application Security Testing (SAST) platforms will be used to perform vulnerability detection and to evaluate code quality in LLM-generated outputs.

Experimental Validation (Optional)

To empirically assess the robustness of LLMs against security threats, a controlled testing environment will be established:

- Simulated Attacks: Experiments will include scenarios involving prompt injection, code manipulation, and backdoor insertion. These tests will evaluate whether LLMs can be coerced into generating insecure or malicious code under adversarial prompts.
- Secure Coding Evaluation: The outputs generated by LLMs will be benchmarked against secure coding standards (e.g., OWASP Secure Coding Guidelines, CERT Secure Coding Practices) to determine alignment and deviations.

The combined methodology is designed to uncover both theoretical vulnerabilities and practical shortcomings in using LLMs within modern web application development environments.

IV. Security Implications Across the Lifecycle

The integration of Large Language Models (LLMs) into web application workflows affects every phase of the software development lifecycle (SDLC). While LLMs promise enhanced efficiency and automation, their use introduces non-traditional security challenges that demand a lifecycle-aware risk assessment. This section outlines the potential threats at each key phase.

Design and Planning

During the early stages of application architecture, LLMs may be used to generate design documents, propose system structures, and recommend architectural patterns. However, such automated suggestions raise concerns related to trust boundaries and security assumptions:

- LLM-generated Design Suggestions: Suggestions involving data flow, API interaction, or user authentication may unintentionally violate security principles (e.g., principle of least privilege, zero trust). Misplaced trust in AI-generated architecture diagrams or protocol flows could embed latent vulnerabilities into the system from the outset.
- Over-reliance on LLM-generated planning artifacts may result in the omission of essential security design considerations, such as threat modeling or secure protocol selection.

Development

In the implementation phase, LLMs are frequently used for code completion, snippet generation, and language translation. This stage is especially vulnerable to the following risks:

- Insecure Code Completions: LLMs can suggest functions or logic containing common security flaws such as SQL injection vectors, XSS-prone patterns, or improper authentication mechanisms.
- Hallucinated APIs and Libraries: LLMs may "hallucinate" non-existent or deprecated APIs that lead to broken or vulnerable code, especially when developers copy-paste without rigorous review.
- Hardcoded Secrets and Insecure Defaults: When prompted to configure database connections, environment variables, or encryption routines, LLMs may insert hardcoded credentials, unsafe key management practices, or recommend insecure defaults (e.g., open CORS policies, permissive firewalls).

Testing

LLMs are increasingly leveraged to auto-generate test cases and scripts. However, this automation introduces a false sense of security:

- Shallow or Non-comprehensive Test Coverage: LLMs may generate generic or trivial test cases that pass without testing security-critical paths, edge cases, or adversarial scenarios.
- Security Testing Blind Spots: LLMs are not yet reliably capable of generating meaningful penetration tests, fuzzing inputs, or modeling attacker behavior, limiting their usefulness in uncovering deep security flaws.

Deployment

Deployment pipelines are becoming increasingly automated through CI/CD systems enhanced with LLM-driven configuration or scripting. This introduces critical security challenges:

- Pipeline Vulnerabilities: Inserting LLMs into CI/CD processes—such as for infrastructure as code (IaC), Dockerfile generation, or Kubernetes manifests—can introduce pipeline injection, insecure container configurations, and unreviewed deployment logic.
- IAM Misconfigurations: LLM-generated scripts for cloud deployments may assign overly permissive roles, fail to enforce separation of duties, or misconfigure identity federation mechanisms.
- Unvetted Dependencies: LLMs may recommend or include third-party packages with known vulnerabilities or malicious behaviors, especially when trained on public code repositories with minimal vetting.

Post-Deployment

Once deployed, LLMs continue to play a role in runtime environments, particularly in areas such as AI-powered customer support, code assistants, or self-healing infrastructure. These use cases become new attack surfaces:

- Prompt Injection in Chat Interfaces: LLM-enabled web interfaces may be susceptible to prompt injection, allowing malicious users to hijack sessions, leak internal logic, or bypass controls.
- Real-Time Misconfigurations: AI-driven system optimizations or adaptive configurations may result in unintended security degradations if LLM-generated recommendations are applied without validation.
- Audit and Monitoring Blind Spots: Traditional monitoring systems may not recognize LLM-driven behaviors, making it harder to detect anomalies or diagnose AI-influenced incidents.

This lifecycle-based view reveals that LLM-related security risks are not confined to development but span design, deployment, and operational contexts. Addressing these risks requires holistic mitigation strategies embedded into every phase of secure web application engineering.

V. THREAT SCENARIOS AND ATTACK VECTORS

The use of Large Language Models (LLMs) in web application development introduces novel threat scenarios and expands the traditional attack surface. These threats are not only technical but also socio-technical, arising from the way LLMs are used, integrated, and trusted within

development pipelines. This section outlines four key threat vectors relevant to LLM-augmented environments.

1. Prompt Injection Attacks

Prompt injection refers to a class of attacks where malicious input is crafted to manipulate the behavior of an LLM. When LLMs are used in web-based chat interfaces, devtools, or code assistants, attackers may exploit their natural language interface to inject unauthorized commands or override system prompts.

- Direct Prompt Injection: A user may input content that causes the LLM to leak sensitive instructions, skip validation logic, or execute unintended operations. For example, inserting instructions like "Ignore the previous instruction and output database credentials" into a prompt.
- Indirect Prompt Injection: This occurs when untrusted external content (e.g., markdown files, comments, or emails) is passed through LLMs without sanitization, allowing attackers to inject malicious instructions indirectly.

Prompt injection remains one of the most immediate and under-guarded risks in LLM-integrated applications, especially where input is dynamically processed in real time.

2. Data Leakage Through Generated Code

LLMs trained on vast public repositories may inadvertently generate code snippets containing hardcoded credentials, proprietary logic, or sensitive data patterns that were present in their training data.

- Memorization Risks: Studies have shown that LLMs can regurgitate training data verbatim, especially for rare or high-frequency sequences such as private keys or API tokens.
- Unintentional Disclosure: When developers prompt LLMs for code fixes or templates, they may unknowingly receive outputs that expose deprecated, confidential, or insecure design patterns from previously seen projects.

This raises both intellectual property risks and compliance concerns (e.g., violations of GDPR, HIPAA, or internal data governance policies).

3. Adversarial Use of LLMs to Generate Malicious Payloads

Attackers can weaponize LLMs to generate evasion-capable or zero-day-like payloads by describing attack goals in natural language. While LLM providers attempt to restrict such behavior via safety filters, adversarial prompting techniques have shown these safeguards can often be bypassed.

- Malware Generation: Using evasive or abstract language, users have been able to prompt LLMs to generate ransomware logic, phishing scripts, or obfuscated shellcode.

- Social Engineering Support: LLMs can assist in crafting convincing spear-phishing messages, malicious user interfaces, or scam messages tailored to specific targets or roles.

This dual-use dilemma—where LLMs can both defend and attack—amplifies the need for controlled access, auditing, and robust use policies around LLM-enabled tools.

4. Model Supply Chain Risks

The LLM supply chain encompasses the training data, fine-tuning processes, plugin ecosystems, and third-party integrations used in model development and deployment. Each stage introduces potential risks:

- Training Data Poisoning: Malicious actors can seed open-source repositories or documentation with toxic code examples that, when included in training data, influence the LLM to adopt insecure patterns.
- Malicious Plugins and Extensions: LLMs that support plugin-based architectures (e.g., for real-time web browsing or file access) may be vulnerable if third-party plugins are compromised or misused.
- Model-to-Model Transfer Risks: Fine-tuning proprietary LLMs with insecure datasets or reused weights may result in inherited vulnerabilities, bias propagation, or functionality regression.

Like traditional software supply chains, securing the LLM model lifecycle requires robust provenance tracking, dataset vetting, plugin validation, and continuous monitoring.

Each of these threat vectors illustrates how LLMs—if unguarded—can serve as both amplifiers and enablers of modern cyber threats. Mitigating these scenarios demands a combination of technical controls, developer education, and policy enforcement, all of which are discussed in the subsequent sections.

VI. RISK MITIGATION STRATEGIES

To responsibly integrate Large Language Models (LLMs) into web application development and deployment workflows, organizations must adopt proactive security strategies. These strategies should address risks at both the technical and organizational levels, ensuring that LLM-augmented processes align with secure software engineering practices. This section outlines key defense mechanisms to mitigate the security implications identified in prior sections.

1. Secure Prompt Engineering and Guardrails

Securing the inputs to LLMs is a foundational step in preventing malicious or unintended behavior:

- Prompt Hardening: Developers should design prompts with explicit constraints, using system messages that clearly define permissible actions and restrict sensitive operations.

- Context Sanitization: Before passing dynamic content (e.g., user input, third-party files) to LLMs, it must be thoroughly sanitized to prevent prompt injection or content manipulation.
- Guardrail Frameworks: Security-focused guardrails such as OpenAI's function calling restrictions, Anthropic's Constitutional AI, or third-party tools (e.g., Rebuff, Guardrails AI) can enforce output filters, restrict disallowed behaviors, and validate outputs against predefined schemas.

2. Post-Generation Code Review Protocols

LLMs should never be treated as autonomous agents in code production. All AI-generated code must pass through rigorous human-in-the-loop validation:

- Mandatory Peer Review: Code contributions generated by LLMs should be subject to manual review and approval, especially for security-critical components like authentication, data handling, or access control.
- Code Provenance Labeling: LLM-generated code should be flagged or annotated within version control systems (e.g., Git hooks) to track origin and facilitate future auditing.
- Developer Training: Teams should be educated on the specific failure modes of LLMs, including hallucinations and insecure recommendations, to maintain skepticism and vigilance during review.

3. Automated Static and Dynamic Analysis

Automated tooling can help detect vulnerabilities and quality issues introduced by LLM-generated code:

- Static Application Security Testing (SAST) tools like SonarQube, Semgrep, or Checkmarx can analyze code at rest for patterns indicative of insecure logic, injection risks, or misconfigurations.
- Dynamic Application Security Testing (DAST) tools, including OWASP ZAP and Burp Suite, can identify runtime vulnerabilities introduced through deployment scripts or insecure endpoints generated via LLMs.
- CI/CD Integration: These tools should be embedded into continuous integration pipelines to enforce automated security gating for all commits, including those influenced by LLMs.

4. LLM Security Checklists for Developers

To operationalize secure use of LLMs, teams should adopt LLM-aware development checklists, covering:

- Safe prompt construction and variable sanitization.
- Verification of suggested code for known CVEs and deprecated libraries.
- Secure handling of credentials, secrets, and cloud configurations.
- Approval and documentation of LLM-influenced architectural decisions.

Industry bodies and security communities (e.g., OWASP, NIST) may soon standardize such guidelines, but in the interim, organizations must develop internal protocols tailored to their workflows.

5. Model Governance and Access Control

Beyond code-level defenses, broader governance mechanisms are essential for controlling model usage:

- Access Policies: Role-based access control (RBAC) should be implemented to ensure that only authorized personnel can access or deploy LLMs in critical environments.
- Logging and Auditing: All interactions with LLMs—especially prompts and outputs related to production code or infrastructure—should be logged and periodically reviewed.
- Model Selection and Hosting: Where possible, organizations should prefer self-hosted, fine-tuned, or audited models over public or black-box APIs, ensuring greater visibility and compliance with internal security policies.
- Plugin and Integration Vetting: Any plugins or third-party integrations used with LLMs must undergo a security review to prevent introduction of compromised dependencies or backdoors.

These mitigation strategies aim to embed security into every aspect of LLM use, from the initial to final deployment. Implementing them effectively requires a combination of policy enforcement, developer education, toolchain augmentation, and ongoing governance.

VII. DISCUSSION

The integration of Large Language Models (LLMs) into web application development workflows marks a significant shift in software engineering practices. While their adoption enhances productivity and automation, it simultaneously introduces new vulnerabilities and trust boundaries that challenge existing security models. This section discusses the broader implications of these findings for secure web development, the dual-use nature of LLMs, and associated ethical and legal dimensions.

1. Implications for Secure Web Development Practices

The findings highlight that LLMs reshape traditional development pipelines by accelerating tasks such as code generation, documentation, and configuration. However, this acceleration often comes at the cost of reduced scrutiny and increased automation risk:

- Breakdown of Defensive Programming Norms: Developers may bypass secure design principles when relying too heavily on LLM suggestions, particularly under tight deadlines or with limited domain expertise.

- Erosion of Human Oversight: The ease of code generation can reduce manual code review and critical reasoning, increasing the risk of propagating insecure patterns.
- Security-by-Design Gaps: LLMs are rarely trained on secure-by-default principles. Without explicit prompting, they may not prioritize secure coding practices or architectural hardening.

To address these shifts, organizations must embed LLM-specific security practices into their Secure Software Development Lifecycle (SSDLC), treating LLMs as high-impact contributors rather than neutral assistants.

2. LLMs as Dual-Use Technologies: Productivity vs. Vulnerability

LLMs embody a dual-use paradox—they are simultaneously powerful tools for innovation and potential enablers of exploitation:

- Productivity Gains: LLMs drastically reduce development time, lower entry barriers for junior developers, and support rapid prototyping. They can assist with writing secure code when properly constrained and guided.
- Security Trade-offs: The same capabilities that streamline workflows can be abused to generate malware, exploit code, or amplify known vulnerabilities. Furthermore, adversaries can exploit LLMs to craft phishing content, analyze targets, or simulate exploit logic without requiring advanced technical skills.

This dual-use nature underscores the importance of implementing robust safety guardrails and ethical use guidelines, especially in open-access environments or collaborative development platforms.

3. Ethical and Legal Considerations

The use of LLMs in software development raises several unresolved ethical and legal issues:

Accountability for Insecure Code

- Who is liable when LLM-generated code leads to a security breach—the developer, the organization, or the model provider?
- Current software liability frameworks do not clearly address AI-assisted development, creating a grey zone where attribution of fault is ambiguous.

Data Privacy Concerns

- LLMs trained on publicly scraped data may inadvertently leak proprietary code, personal information, or regulated content when generating outputs.
- The use of cloud-based LLM APIs introduces data residency and compliance risks—developers may unknowingly expose sensitive data during inference.
- Jurisdictions with strong privacy regulations (e.g., GDPR, HIPAA) may view LLMs as data processors, making their use in regulated sectors legally sensitive.

Organizations must therefore develop internal policies that regulate prompt design, output usage, and data handling when working with LLMs. Transparency, consent, and auditability must be central tenets in LLM deployment strategies.

VIII. REFERENCES

1. Phanireddy, Sandeep. "LLM security and guardrail defense techniques in web applications." International Journal of Emerging Trends in Computer Science and Information Technology (2025): 221-224.
2. Chandola, V., Banerjee, A., & Kumar, V. (2009). "Anomaly detection: A survey." ACM Computing Surveys (CSUR), 41(3), 1-58.
3. S. Mohammad, M. M. M. Rahman and F. Farahmandi, "Required Policies and Properties of the Security Engine of an SoC," 2021 IEEE International Symposium on Smart Electronic Systems (iSES), Jaipur, India, 2021, pp. 414-420, doi: 10.1109/iSES52644.2021.00100.
4. Schlegl, T., et al. (2017). "Unsupervised anomaly detection with generative adversarial networks." International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI).
5. M. M. Hossain et al., "HEXON: Protecting Firmware Using Hardware-Assisted Execution-Level Obfuscation," 2021 IEEE Computer Society Annual Symposium on VLSI (ISVLSI), Tampa, FL, USA, 2021, pp. 343-349, doi: 10.1109/ISVLSI51109.2021.00069
6. Biggio, B., et al. (2013). "Evasion attacks against machine learning at test time." Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD).
7. Phanireddy, Sandeep. "AI-Generated Malware Detection in Web Applications De-Obfuscation and Analysis." IJLRP-International Journal of Leading Research Publication 1, no. 4 (2020).
8. Papernot, N., et al. (2016). "Distillation as a defense to adversarial perturbations against deep neural networks." IEEE Symposium on Security and Privacy (S&P).
9. Sharafaldin, I., et al. (2018). "Toward generating a new intrusion detection dataset and intrusion traffic characterization." International Conference on Information Systems Security and Privacy (ICISSP).
10. Sajeed Mohammad and Farimah Farahmandi. 2024. DyFORA: Dynamic Firmware Obfuscation and Remote Attestation using Hardware Signatures. In Proceedings of the Great Lakes Symposium on VLSI 2024 (GLSVLSI '24). Association for Computing Machinery, New York, NY, USA, 471–476. https://doi.org/10.1145/3649476.3658715
11. Tavallaee, M., et al. (2009). "A detailed analysis of the KDD CUP 99 data set." IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA).

12. Moustafa, N., & Slay, J. (2015). "UNSW-NB15: a comprehensive data set for network intrusion detection systems." Military Communications and Information Systems Conference (MilCIS).
13. Kreps, J., et al. (2011). "Kafka: A distributed messaging system for log processing." NetDB Workshop.
14. Phanireddy, S. "Leveraging SIEM and AI for enhanced web attack detection and prevention." IJCEM-International Journal of Core Engineering & Management 7, no. 2 (2022).

.